Relatório Técnico - Sincronização de Metrô (METROREC)

Implementação 4: Concorrência e Sincronização com Pthreads

Instituição: CESAR School	**Disciplina:** Sistemas Operacionais (2026.2)
Aluno: Leonardo Gutzeit Borgmann Matos	**E-mail / Login:** lgbm@cesar.school (**lgbm**)

1. Objetivo do Projeto

O objetivo deste projeto é projetar e implementar um mecanismo de sincronização concorrente em linguagem C para o sistema automatizado de vagões e embarque de passageiros da METROREC, derivado do clássico problema da Caltrain. Cada vagão e cada passageiro são gerenciados por threads independentes. O sistema deve atender às seguintes restrições obrigatórias:

- Utilização estrita de primitivas POSIX Pthreads (mutex e variáveis de condição).
- Proibição estrita de semáforos ou outras primitivas de sincronização externas.
- **Uso de no máximo um único lock (mutex)** em cada struct station.
- Permitir **embarque simultâneo** (Requisito 5): diversos passageiros chamam e retornam de station_wait_for_car antes que qualquer um deles chame station_on_board.
- Ausência total de espera ocupada (*busy-waiting*), bloqueando threads via pthread_cond_wait.
- Automatização de compilação via Makefile, gerando o executável sync_car.

2. Modelagem da Estrutura e Primitivas

Para cumprir com rigor a restrição de utilizar um único lock por estação, a estrutura struct station foi concebida com um único mutex e duas variáveis de condição:

```
struct station {
    pthread_mutex_t lock;           /* Único mutex permitido por estação */
    pthread_cond_t train_arrived;   /* Sinaliza chegada de vagão com vagas */
    pthread_cond_t train_loaded;    /* Sinaliza término do embarque para o vagão */
    int waiting_passengers;         /* Passageiros aguardando na plataforma */
    int available_seats;            /* Assentos livres no vagão presente */
    int boarding_passengers;        /* Passageiros autorizados em trânsito de embarque */
};
```

A variável **boarding_passengers** é o elemento-chave da modelagem: ela desacopla a alocação da vaga (que ocorre em station_wait_for_car) da confirmação de acomodação (em station_on_board), possibilitando que múltiplos passageiros embarquem simultaneamente sem que o trem parta prematuramente.

3. Detalhamento da Implementação

3.1 station_load_car: Chamada quando o vagão atraca na estação com count vagas. A função armazena as vagas, emite um broadcast (pthread_cond_broadcast) para acordar os passageiros e aguarda em pthread_cond_wait(&station-;>train_loaded, &station-;>lock) enquanto houver vagas e passageiros esperando, ou enquanto houver passageiros em embarque ativo (boarding_passengers > 0). Ao sair do laço, zera as vagas e parte.

3.2 station_wait_for_car: Chamada ao passageiro chegar à estação. Incrementa waiting_passengers e dorme em pthread_cond_wait(&station->train_arrived, &station->lock) enquanto available_seats == 0. Ao obter vaga, decrementa waiting_passengers e available_seats, incrementa boarding_passengers, libera o mutex e retorna imediatamente.

3.3 station_on_board: Chamada quando o passageiro senta. Decrementa boarding_passengers. Se este for o último passageiro em trânsito (boarding_passengers == 0) e a condição de partida foi atingida (vagão cheio ou fila vazia), sinaliza o vagão através de pthread_cond_signal(&station->train_loaded).

4. Análise de Concorrência e Corretude

Propriedade	Mecanismo de Garantia
Ausência de Busy-Waiting	Bloqueio atômico de threads via pthread_cond_wait, sem laços vazios.
Embarque Simultâneo	O mutex é liberado antes da chamada de station_on_board; múltiplos passageiros retornam concorrentemente de station_wait_for_car.
Proteção contra Spurious Wakeups	Todas as verificações de condição utilizam laços while protetores.
Prevenção de Deadlock	Existe exatamente 1 mutex no sistema, eliminando qualquer grafo cíclico de espera.
Integridade dos Assentos	Alocação atômica protegida pelo mutex; nenhum vagão parte com assentos negativos.

5. Metodologia de Testes e Resultados

Para validar a implementação, foi desenvolvida uma suíte com 7 testes cobrindo todas as rubricas no arquivo main.c:

ID	Cenário de Teste	Condição Avaliada	Status
T1	Capacidade == Demanda	10 passageiros, 1 carro de 10 vagas	**PASS**
T2	Vagas > Demanda	6 passageiros, 1 carro de 15 vagas (parte quando esvazia)	**PASS**
T3	Vagas < Demanda	10 passageiros, 1 carro de 4 vagas (6 sobram na fila)	**PASS**
T4	Múltiplos Carros	15 passageiros atendidos por 3 carros (4, 5 e 6 vagas)	**PASS**
T5	Estação Vazia	0 passageiros, carro de 10 vagas parte imediatamente	**PASS**
T6	Embarque Simultâneo	5 passageiros retornam de wait_for_car antes do on_board	**PASS**
T7	Stress Test Concorrente	50 passageiros e 4 vagões sob rajadas concorrentes	**PASS**

Taxa de Aprovação nos Testes: 7 de 7 aprovados (100% de sucesso).

6. Dificuldades Encontradas e Soluções

• **Sincronização do Embarque Simultâneo:** A maior complexidade residiu em permitir o retorno de múltiplos passageiros em station_wait_for_car antes que qualquer um deles chamasse station_on_board, sem gerar condição de corrida na partida do trem. A solução adotada foi o contador boarding_passengers, assegurando que o vagão nunca parta com passageiros em trânsito.

• **Partida com Estação Vazia:** Quando um vagão chega com mais assentos que passageiros, ele deve partir assim que os passageiros presentes sentarem. Isso foi solucionado pela condição (available_seats == 0 || waiting_passengers == 0) && boarding_passengers == 0.

7. Conclusão

O sistema de sincronização desenvolvido para o METROREC cumpre integralmente todas as rubricas especificadas. A modelagem com apenas 1 mutex e 2 variáveis de condição demonstrou-se robusta, enxuta e eficiente, garantindo embarque simultâneo, ausência de busy-wait e total prevenção contra deadlocks e condições de corrida.